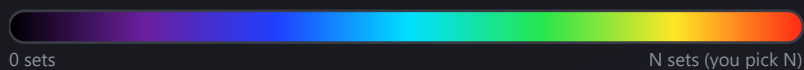


hotMuscles

A phone app for logging gym workouts. It shows how many sets each muscle got in the last 7 days as a colour heatmap on a body.



884

exercises

30

muscle regions

5

screens

4

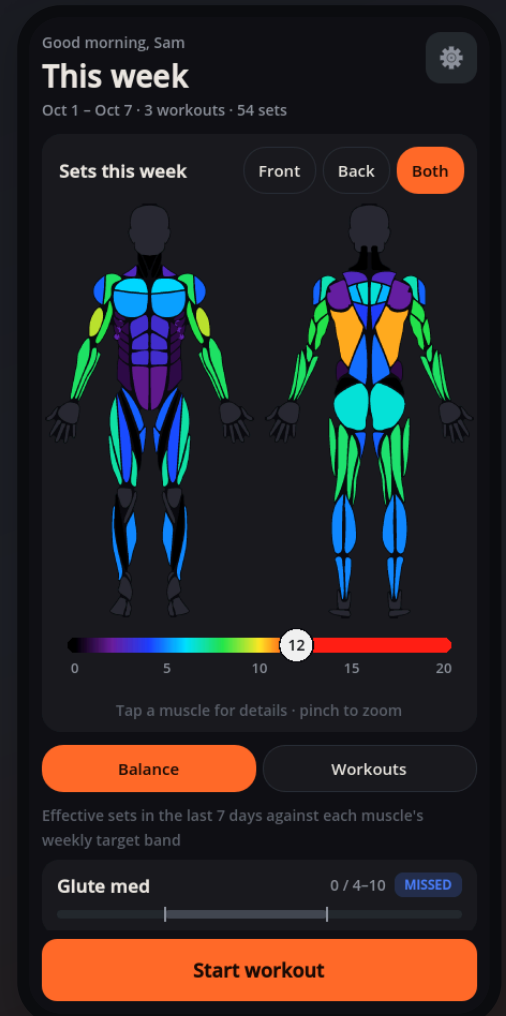
bodies × views

4 · 3

themes · gradients

8.1k

lines of app code



The home screen: this week's heat on front + back

What's inside

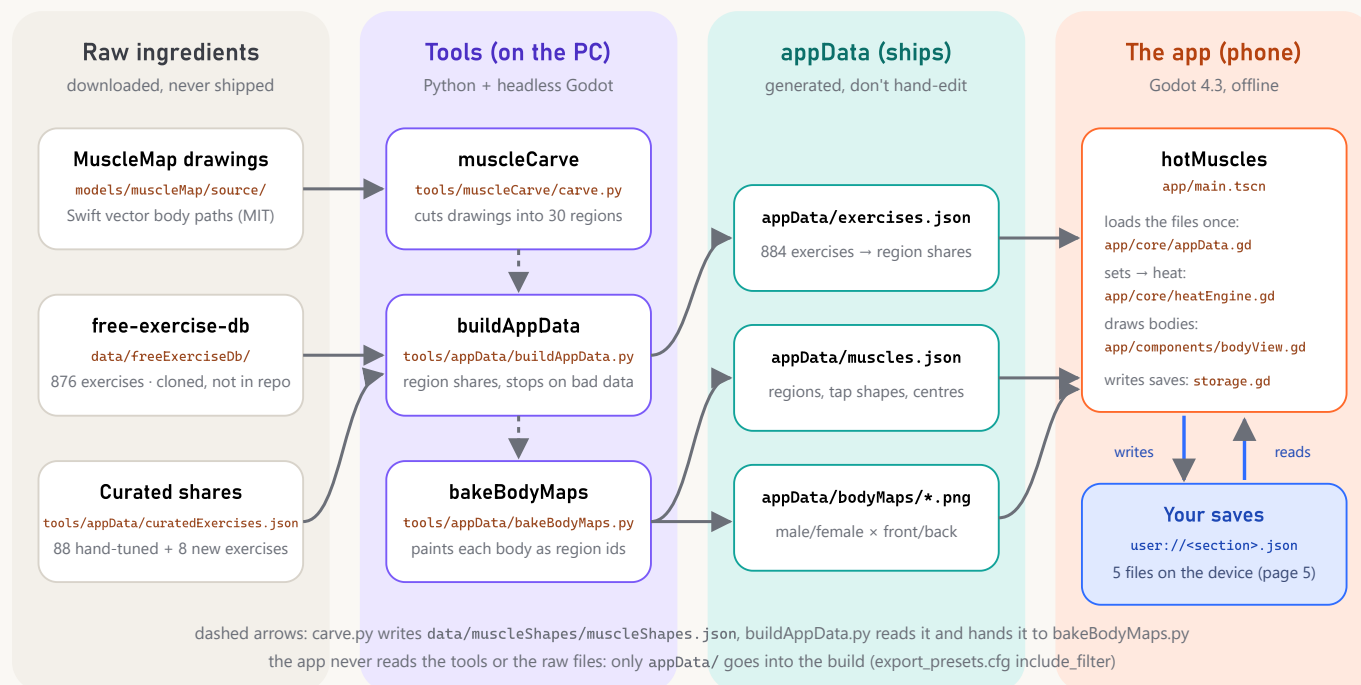
1 Data flow · 2 App structure · 3 Screens and navigation · 4 Saved data · 5 Heat calculation · 6 Body rendering · 7 Behind the scenes

Built with Godot 4.3 · one codebase → Android apk + Windows exe · fully offline

Reading the file names: every `monospace path` is a real file, counted from the top of the repo (github.com/aphyr-dev/hotMuscles). A name ending in `()` is a function inside the file named next to it.

Data flow: from source files to the app on your phone

Data moves left to right. Body drawings and an exercise list go in on the left. Python tools on the PC turn them into data files, which are packed into the app and read when it starts. The only thing the app writes is your own workout log, saved on the phone.



tools run on the PC only

Re-run `python`
`tools/appData/buildAppData.py`
when a body drawing or an exercise share changes. If a share is out of range, an id is unknown, or a muscle can't be reached, it stops and writes nothing.

appData generated, ships

Finished data files, read by
`AppData.loadAll()` in
`app/core/appData.gd`. The body PNGs ship exactly as made: each has a `.png.import` set to "keep", so Godot never re-compresses them.

saves written by the app

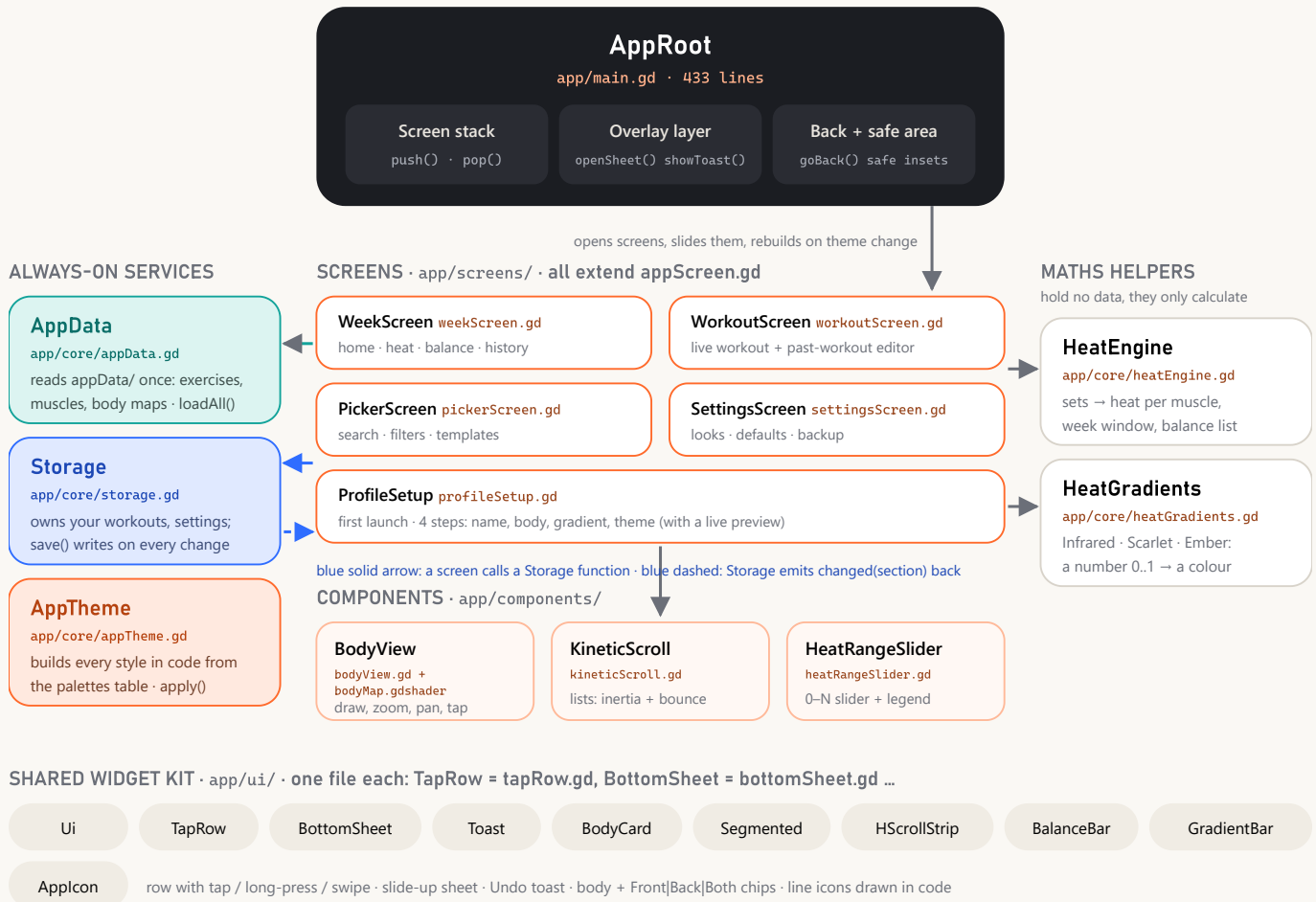
The only files the app writes, all through `app/core/storage.gd`. Small, versioned, crash-safe. Settings → "Export backup" writes them all into one backup file.

Project folders and the files in them

FOLDER	SHIPS?	FILES
app/	in the app	main.gd + main.tscn the root · core/ appData, storage, appTheme, heatEngine, heatGradients · screens/ appScreen (shared base), weekScreen, workoutScreen, pickerScreen, settingsScreen, profileSetup · components/ bodyView + bodyMap.gdshader, kineticScroll, heatRangeSlider · ui/ ten small widgets
appData/	in the app	exercises.json · muscles.json · bodyMaps/male_front.png, male_back, female_front, female_back
tools/	PC only	appData/ buildAppData.py, bakeBodyMaps.py, curatedExercises.json · muscleCarve/ extractShapes.py, carve.py, polygonCut.gd + preview/inspection helpers · build/ buildAll.py, checkBuild.py · icon/ makelcon.py
tests/	PC only	runGodot.py runs them all · test*.gd checks · shot*.gd screenshot drivers (page 8)
data/ · models/	PC only	data/muscleShapes/ sourceShapes.json + muscleShapes.json · data/sources.md where every input came from · models/muscleMap/source/ the MuscleMap Swift files
docs/ · icons/	PC only	docs/appSpec.md the spec · this guide · icons/ launcher PNGs made by makelcon.py · root: project.godot, export_presets.cfg

App structure: main parts and how they connect

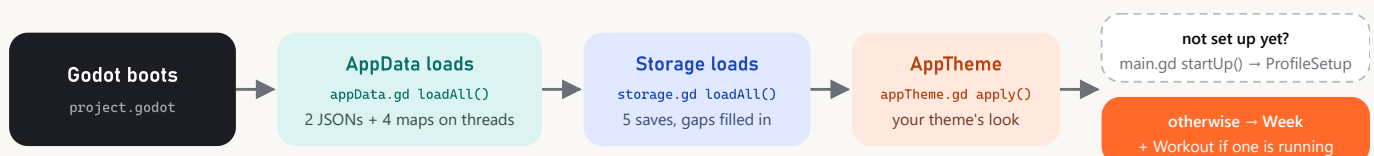
`AppRoot` (`app/main.gd` , the script on `app/main.tscn`) is the top node of the app. It keeps the stack of open screens and the layer for sheets and toasts. Three services stay loaded the whole time (Godot "autoloads", listed in `project.godot`): `AppData` holds the shipped files, `Storage` holds your saves, `AppTheme` holds the colours. Two helpers, `HeatEngine` and `HeatGradients`, only calculate. Screens are built from shared widgets in `app/ui/` .



Update loop: a screen calls a Storage function such as `Storage.setEntrySets()` . That ends in `save()` in `storage.gd` , which writes the file and emits `changed("workouts")` (a Godot signal: a broadcast message). Each screen connects to it in its `_ready()` and refreshes in `_onStorageChanged()` . Screens never call each other directly.

Theme change: `AppTheme.apply()` rebuilds every style from the `palettes` table at the top of `appTheme.gd` (page, surface, text, accent, cold, hot, ghost...) and emits `themeChanged` . `_onThemeChanged()` in `main.gd` then calls `rebuild()` on every open screen.

Start-up sequence



Limit: under 500 ms from the main scene to the first drawn frame. `_onFirstFrame()` in `main.gd` prints the time, and the build check reads it (page 8).

```

graph LR
    ProfileSetup[ProfileSetup  
profileSetup.gd  
first launch only] -.->|finish| Week[Week  
weekScreen.gd  
home · always at the bottom of the stack]
    Week -->|openWorkout()| WorkoutLive[Workout (live)  
workoutScreen.gd createLive()  
timer · stepper · finish]
    Week -->|openWorkoutEditor()| WorkoutEditor[Workout (editor)  
workoutScreen.gd createEditor()  
same screen, past workout]
    Week -->|openSettings()| Settings[Settings  
settingsScreen.gd  
looks · defaults · backup]
    Week -->|openRegion()| RegionSheet[Region sheet  
weekScreen.gd openRegion()]
    WorkoutLive --> Picker[Picker  
pickerScreen.gd  
add exercises]
    WorkoutEditor --> Picker
    RegionSheet -.->|Run setup again  
openProfileSetup()| ProfileSetup
    RegionSheet -.->|Find exercises -> openPicker(target, regionId), filtered to that muscle| Settings
    Settings -.-> Picker
  
```



- Back button: `main.gd goBack()`, in order

- Resume and edit:** closed mid-workout? `startUp()` in `main.gd` opens Week, then the running workout on top. The editor changes a copy until "Save changes" calls `updateWorkout()`.

Saved data: the five save files and their fields

Each kind of data is saved in its own JSON file in Godot's `user://` folder, all written by `app/core/storage.gd`. `loadAll()` reads all five at start-up and `save()` rewrites one whenever its data changes. Every file is wrapped as `{"version": 1, "data": ...}`, so a future layout change can upgrade old files (`_migrate()`).

Where `user://` is: Android `/data/data/com.aphyr.hotmuscles/files/` (private to the app; uninstalling deletes it) · Windows `%APPDATA%\Godot\app_userdata\hotMuscles\` · backups go in `user://backups/backup_<date-time>.json` plus the clipboard.

profile	profile.json	settings	settings.json	prefs	exercisePrefs.json
defaults: <code>defaultProfile()</code>		defaults: <code>defaultSettings()</code>		per exercise id · defaults: <code>defaultPref()</code>	
name	text	hideUntouched	yes / no	favourite	star in the picker
body	male female	newExerciseSets	0 or 1 (default 1)	hidden	hide from lists
gradient	infrared scarlet ember	defaultView	front back both	grips	remembered checkbox
theme	ember ocean forest light	rangeWeek	N for the week · 12	lastUsed	time, for sorting
setupDone	yes / no	rangeWorkout	N for a workout · 5		

workouts.json: a workout is a list of entries, an entry is one exercise and its sets

workouts.json	{ submitted: [every finished workout], current: the running one, or null }
workout id · startedAt · endedAt (0 = still running) entry exerciseld "Face Pull" sets 3 grips false sets 0 = planned, shown dashed, dropped on finish entry exerciseld "Bent Over Barbell Row" sets 4 grips true grips ticked = straps used, so no forearm heat from this one ... more entries	templates.json a list of { id, name, entries } same entry shape, so a template drops straight into a workout
	backup file all five sections in one JSON, plus the app name, backupVersion, exportedAt <code>storage.gd exportBackup()</code>

Which screen writes which file

FILE	WRITTEN BY	STORAGE FUNCTIONS CALLED
profile.json	profileSetup.gd, settingsScreen.gd	setProfileValues() · setProfile()
settings.json	settingsScreen.gd, the 0–N slider in <code>app/ui/bodyCard.gd</code>	setSetting()
exercisePrefs.json	pickerScreen.gd (star, hide), workoutScreen.gd (grips)	setFavourite() · setHidden() · setGripsMemory()
workouts.json	weekScreen.gd, workoutScreen.gd, pickerScreen.gd	startWorkout() · addEntries() · setEntrySets() · finishWorkout() · deleteWorkout() · updateWorkout()
templates.json	week + workout (save), picker (rename, delete)	saveTemplate() · renameTemplate() · deleteTemplate()

Crash-safe writing: `_writeSection()` writes `workouts.json.tmp` first and swaps it in only once it is complete, so a power cut mid-save leaves the old file whole. A file that can't be read is renamed `workouts.json.bad` by `_readSection()` and only that section starts fresh.

Backup import: `isBackup()` checks every section has the right type. `askImport()` in `settingsScreen.gd` shows the workout and template counts and asks you to confirm. Then `exportBackup()` saves your current data first and `importBackupText()` replaces everything. All or nothing.

Heat calculation: from exercises to muscle colours

Each exercise in `appData/exercises.json` lists the muscle regions it works and how much of a set counts for each (its **shares**). `HeatEngine.weekHeat()` in `app/core/heatEngine.gd` multiplies your sets by the shares and adds them up over the last 7 days. `HeatGradients.amountColour()` turns each region's total into a colour.

exercise record

`appData/exercises.json`

id · name "Ab_Roller" · Ab Roller
equipment · category 12 kinds · 7 kinds
forearmKind none | grip | direct
curated yes = hand-tuned shares

targets: one share per region (1.0 = a full set)

upperAbs		1.0
lowerAbs		0.8
serratus		0.4
lats		0.3
obliques		0.3

Where the shares come from

Automatic (788): `tools/appData/buildAppData.py` reads the coarse tags in

`data/freeExerciseDb/dist/exercises.json` (primary 1.0, secondary 0.5) and splits each group over our regions with its `defaultSpread` table (chest → lowerChest 0.7, upperChest 0.3).

Curated (96): typed by hand in

`tools/appData/curatedExercises.json`. **Stretching, cardio (137):** no heat.

Grips: 106 exercises have `forearmKind` "grip":

`HeatEngine.entryHeat()` skips their forearm share when you tick grips. 28 are "direct" and always count.

$$\text{heat[muscle]} = \sum \text{sets} \times \text{share}$$

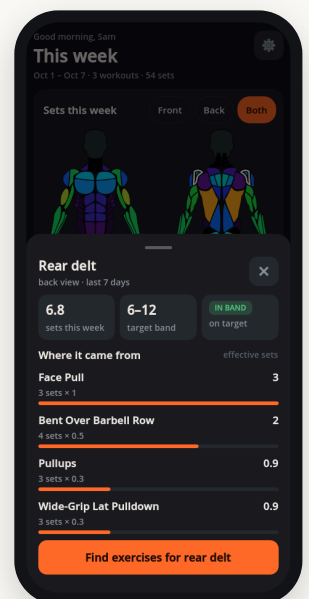
`weekHeat()` = `workoutsInWindow()` → `effectiveSets()` → `entryHeat()` per entry

Worked example: rear delt (same numbers as the screenshot →)

Face Pull	3 sets × 1.0		3.0
Bent Over Barbell Row	4 sets × 0.5		2.0
Pullups	3 sets × 0.3		0.9
Wide-Grip Lat Pulldown	3 sets × 0.3		0.9
rear delt this week			6.8

band 6–12 → status "ok"

The face pull works the rear delt fully (1.0); the row and the lat exercises only assist (0.5, 0.3). Tapping a muscle opens the region sheet (`openRegion()` in `weekScreen.gd`), which lists these rows from `HeatEngine.contributions()`.



Region sheet for the rear delt

Last 7 days, no fading

`workoutsInWindow()` keeps the 7 × 24 h before now, not the calendar week. Older workouts drop out.

Heat number to colour

`amountColour()` samples $t = \text{sets} \div N$ on the `presets` table in `heatGradients.gd`. The slider sets N; above N is the top colour.

Status vs target band

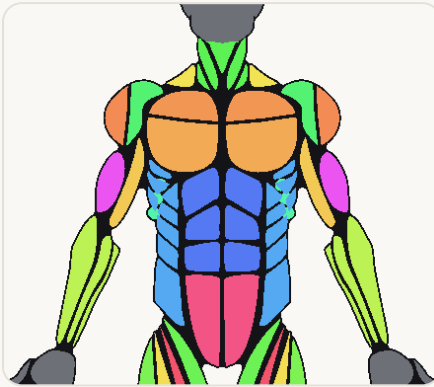
missed under ok over

`targetStatus()` checks each region's `band` in `muscles.json` (from `targetBands` in `buildAppData.py`; 6–12 for most). Under 0.5 sets is missed. `rankRegions()` sorts the balance list.

30 regions in 7 groups (`regions`, `groups` in `appData/muscles.json`): Back 6 · Shoulders 4 · Chest 3 · Arms 3 · Core 4 · Legs 9 · Neck 1. The picker ranks by `HeatEngine.recommend()`: $\text{share} + 0.5 \times \text{focus}$, plus small bonuses for favourites and used exercises.

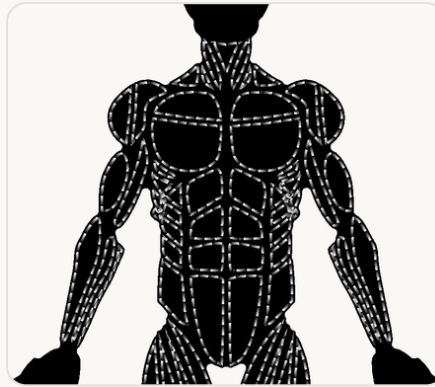
Body rendering: region map texture, palette and shader

Each body is drawn as one textured rectangle by `app/components/bodyView.gd`. The texture is not a picture. It is a region map, `appData/bodyMaps/<body>_<view>.png`, baked by `tools/appData/bakeBodyMaps.py`: each colour channel stores a different number per pixel (like an ID pass in compositing). The shader `app/components/bodyMap.gdshader` reads those numbers and colours the body on every frame. Below: `male_front.png` split into its channels.



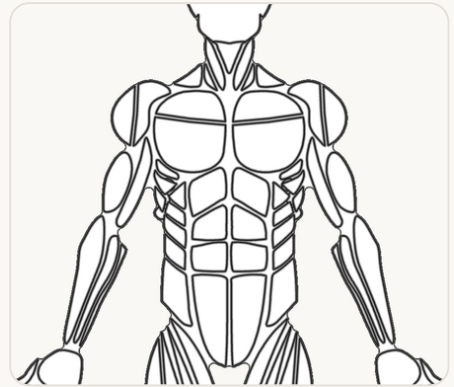
R · which region

the id of the muscle at this pixel: 0 outside the body, 1 seam between regions, 2 head, hands and feet, 3 and up one of the 30 regions (the table is `mapIds` in `muscles.json`). Shown with a random colour per id.



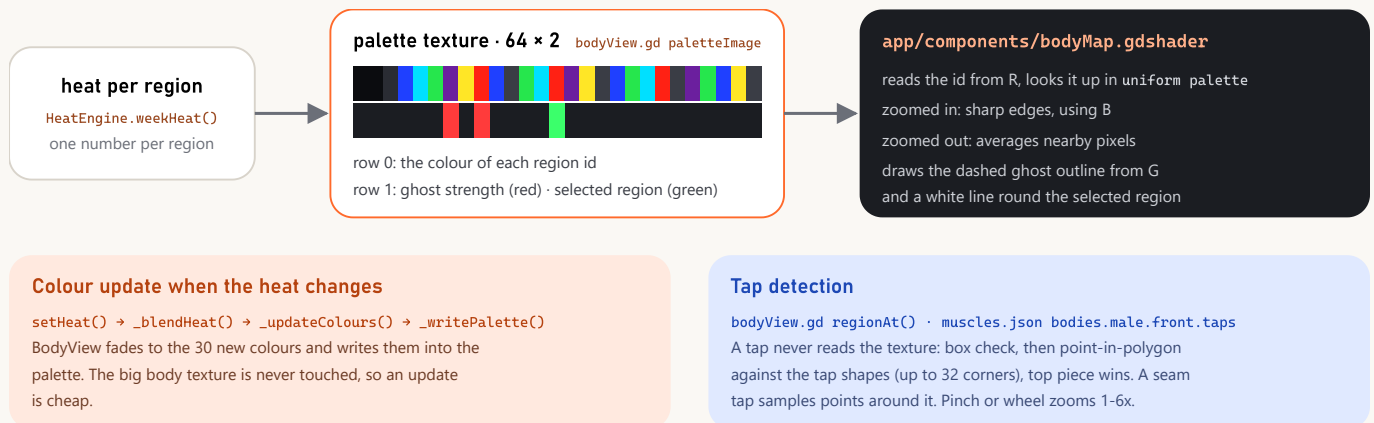
G · position along the outline

A number that runs along each muscle's edge. The shader uses it to draw the dashed "ghost" outline that marks exercises you planned but have not done yet.



B · distance to the edge

How far the pixel is from the nearest edge. The shader uses it to keep muscle edges sharp at any zoom level.



From a logged workout to colours on the Week screen

1 · Save

`Storage.save()` writes
`user://workouts.json`
 and emits
`changed("workouts")`.

2 · Count

`_onStorageChanged()` in
`weekScreen.gd` calls
`HeatEngine.weekHeat()`:
 one number per region.

3 · Colour

`bodyCard.setHeat() →`
`bodyView.setHeat();`
 each number goes
 through `amountColour()`
 into the palette.

4 · Draw

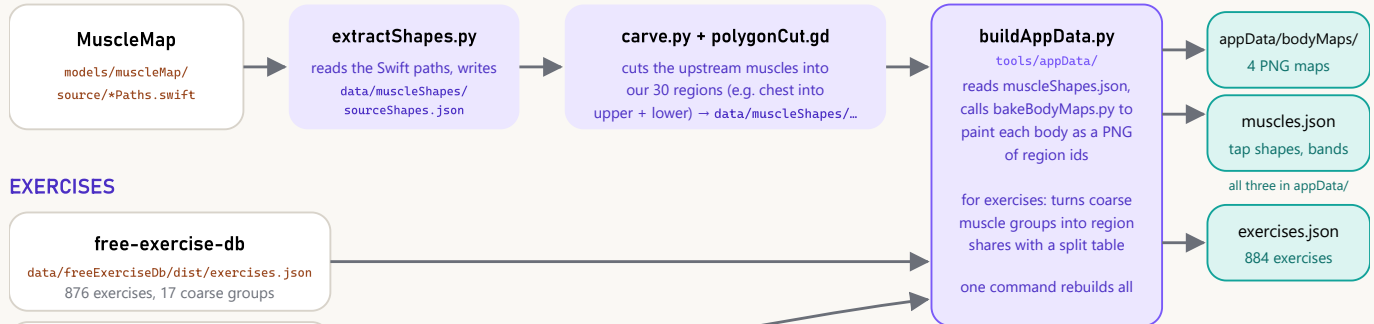
`bodyMap.gdshader` looks
 up every pixel's region id
 in the palette, so the GPU
 recolours the body.

Why it works this way: an earlier version drew tens of thousands of outline points per body on every frame. Now the GPU gets one rectangle and two small textures, and the phone builds no geometry at all. If a body map file is missing, `AppData.bodyMap()` logs an error naming the file instead of quietly drawing something else.

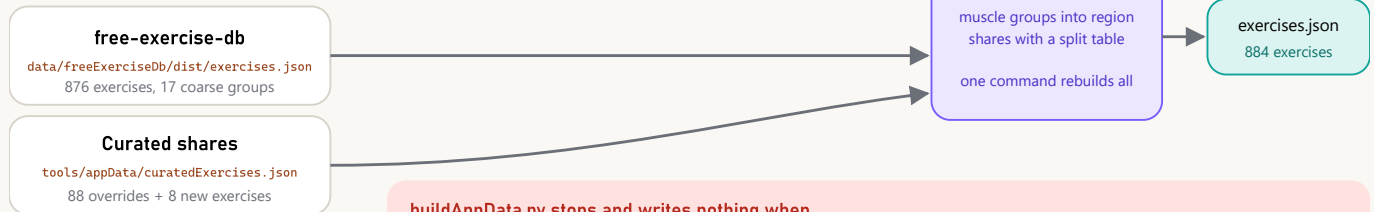
Behind the scenes: building the data and keeping it fast

Two jobs happen out of sight. Scripts in `tools/` build the data files once on the PC, so the phone has almost nothing to compute at start-up. And every build is checked against two time limits, so the app stays quick to open and smooth to use.

BODY SHAPES



EXERCISES



buildAppData.py stops and writes nothing when...

an exercise or region id is unknown · a share is outside 0-1 · any muscle can't be reached by some exercise

< 500 ms

main scene → first frame, printed by `_onFirstFrame()` in `main.gd`, checked by `tests/testStartup.gd` and `checkBuild.py` (fresh-PC first launch: under 2 s)

< 40 ms

slowest frame in any everyday step (tap, scroll, open a screen): `worstFrameLimitMs` in `testStartup.gd`. Aim 33 ms (30 fps); v0.3 worst was 27–32 ms

How the app meets those limits

- ✓ **Long lists fill in over several frames.** `pickerScreen.gd` builds `firstRows` (6), then `rowsPerFrame` (3) per frame up to `pageSize` (40); all at once was a 90 ms stutter. `weekScreen.gd` does the same (`firstBalanceRows`).
- ✓ **Update rows, don't rebuild them.** Existing rows are changed or moved in place.
- ✓ **One write for many values.** A write costs about 4 ms, so `Storage.addEntries()` and `setProfileValues()` save many values at once.
- ✓ **Body maps load in the background.** `_startMapDecodes()` in `appData.gd` decodes the PNGs on `ThreadPool` threads.
- ✓ **Idle bodies sleep.** `bodyView.gd` calls `set_process(false)` once nothing on the body is animating.

Automatic checks in `tests/`, all run through `runGodot.py`: `testHeatEngine.gd` (maths), `testStorage.gd` (saves, backups), `testFlows.gd` (every screen with real numbers), `testTouchUi.gd` (synthetic finger), `testKineticScroll.gd`, `testStartup.gd` (timing), `testBuildAppData.py` (bad data refused). `checkBuild.py` also opens each exe three times on a throwaway save folder.

Rebuild and ship

```
python tools/appData/buildAppData.py
python tools/build/buildAll.py
python tools/build/checkBuild.py <exe> <apk>
```

Builds land in `builds/hotMuscles_vNNN` (not in the repo; shared on the Releases page). `checkBuild.py` fails any build that holds `data/`, `models/`, `tools/`, `tests/`, `docs/` or `icons/`.